ARQUIVOS TEXTO (.TXT) EM C

Abrir, ler, pesquisar, escrever e acrescentar dados em arquivos

Disciplina: Algoritmos II

Prof. Dr. Luã Marcelo Muriana

Material didático com exemplos completos em C[1]

Objetivo do material: ao final, você deverá conseguir criar um programa em C que abra um arquivo .txt, leia seu conteúdo, procure informações, grave novos dados e acrescente informações sem apagar o conteúdo existente.

[1] Este material foi gerado com auxilio da Inteligência Artificial.

1. O que é um arquivo texto?

Até aqui, muitos programas armazenam dados apenas em variáveis e vetores. Esses dados existem enquanto o programa está em execução. Quando o programa termina, o conteúdo dessas variáveis deixa de estar disponível. Um arquivo permite armazenar informações de forma persistente.

Um arquivo texto (.txt) armazena caracteres que podem ser lidos por programas e também por editores de texto. Em C, o programa não manipula o arquivo diretamente: ele utiliza um ponteiro do tipo FILE para representar a conexão entre o programa e o arquivo.

```
#include <stdio.h>

FILE *arquivo;
```

Importante: FILE é um tipo definido pela biblioteca stdio.h. Por isso, para trabalhar com arquivos nos exemplos desta aula, utilizaremos #include <stdio.h>.

1.1 Fluxo básico de trabalho

1. Declarar um ponteiro do tipo FILE.
2. Abrir o arquivo com fopen().
3. Verificar se a abertura foi realizada com sucesso.
4. Ler ou escrever os dados.
5. Fechar o arquivo com fclose().

```
FILE *arquivo;

arquivo = fopen("dados.txt", "r");

if (arquivo == NULL) {
    printf("Nao foi possivel abrir o arquivo.\n");
    return 1;
}

/* leitura ou escrita */

fclose(arquivo);
```

1.2 Por que fechar o arquivo?

O fechamento informa ao sistema que o programa terminou de utilizar o arquivo. Além de liberar recursos, ele é especialmente importante na escrita, pois dados que ainda estejam em buffers podem ser efetivamente enviados ao arquivo durante o fechamento.

2. Abrindo um arquivo com fopen()

A função fopen() recebe duas informações principais: o nome (ou caminho) do arquivo e o modo de abertura.

```
arquivo = fopen("dados.txt", "r");
```

Nesse exemplo, dados.txt é o arquivo e "r" indica que ele será aberto para leitura.

Modo	Finalidade	Efeito principal
"r"	Leitura	O arquivo precisa existir.

"w"	Escrita	Cria o arquivo ou apaga o conteúdo anterior.
"a"	Acrescentar	Cria se necessário e escreve no final, preservando o conteúdo.
"r+"	Leitura e escrita	O arquivo precisa existir.
"w+"	Leitura e escrita	Cria ou apaga o conteúdo anterior.
"a+"	Leitura e acréscimo	Preserva o conteúdo e escreve no final.

Atenção: abrir um arquivo existente com "w" apaga seu conteúdo anterior. Para adicionar informações ao final sem apagar o que já existe, utilize "a".

2.1 Verificando se fopen() funcionou

Se a abertura falhar, fopen() retorna NULL. Portanto, sempre verifique o resultado antes de tentar utilizar o arquivo.

```
FILE *arquivo = fopen("dados.txt", "r");

if (arquivo == NULL) {
    printf("Erro ao abrir o arquivo.\n");
    return 1;
}
```

A abertura pode falhar, por exemplo, porque o arquivo não existe no modo de leitura, porque o caminho está incorreto ou porque o programa não possui permissão para acessar o local.

3. Lendo um arquivo texto

Há diferentes maneiras de ler um arquivo. A escolha depende de como os dados estão organizados.

3.1 Leitura caractere por caractere: fgetc()

fgetc() lê um caractere por vez. Ela é útil quando queremos analisar o conteúdo em nível de caracteres.

```
#include <stdio.h>

int main() {
    FILE *arquivo;
    int c;

    arquivo = fopen("texto.txt", "r");

    if (arquivo == NULL) {
        printf("Erro ao abrir o arquivo.\n");
        return 1;
    }

    while ((c = fgetc(arquivo)) != EOF) {
        printf("%c", c);
    }

    fclose(arquivo);
    return 0;
}
```

A constante EOF (End Of File) indica que não há mais caracteres para serem lidos. A variável c é int porque fgetc() precisa representar tanto todos os valores possíveis de caracteres quanto o valor especial EOF.

3.2 Leitura linha por linha: fgets()

Para arquivos organizados em linhas, fgets() costuma ser uma escolha muito conveniente. Ela lê uma linha (ou até atingir o limite do vetor de caracteres).

```
#include <stdio.h>

int main() {
    FILE *arquivo;
    char linha[200];

    arquivo = fopen("texto.txt", "r");

    if (arquivo == NULL) {
        printf("Erro ao abrir o arquivo.\n");
        return 1;
    }

    while (fgets(linha, 200, arquivo) != NULL) {
        printf("%s", linha);
    }

    fclose(arquivo);
    return 0;
}
```

- linha é o vetor em que o texto lido será armazenado.

- 200 é a capacidade do vetor.
- arquivo indica de onde os dados serão lidos.
- Quando não há mais linhas para ler, fgets() retorna NULL.

3.3 Leitura formatada: fscanf()

fscanf() funciona de forma semelhante a scanf(), mas lê os dados de um arquivo. É útil quando o arquivo possui uma estrutura conhecida.

Considere o arquivo notas.txt:

```
Ana 8.5
Bruno 6.0
Carlos 9.2
```

Podemos ler cada nome e nota assim:

```
#include <stdio.h>

int main() {
    FILE *arquivo;
    char nome[50];
    float nota;

    arquivo = fopen("notas.txt", "r");

    if (arquivo == NULL) {
        printf("Erro ao abrir o arquivo.\n");
        return 1;
    }

    while (fscanf(arquivo, "%49s %f", nome, &nota) == 2) {
        printf("Aluno: %s | Nota: %.1f\n", nome, nota);
    }

    fclose(arquivo);
    return 0;
}
```

Por que == 2? A leitura espera dois itens: um texto e um número real. O laço continua enquanto fscanf() conseguir ler corretamente os dois itens.

4. Pesquisando informações no arquivo

Pesquisar em um arquivo texto significa percorrer seu conteúdo e verificar se o dado desejado aparece. Em arquivos pequenos e sem uma estrutura especial de índice, uma estratégia simples é realizar uma busca sequencial.

4.1 Procurando um número

Considere numeros.txt:

```
15 28 7 42 10 33
```

O programa abaixo procura um número informado pelo usuário:

```c
#include <stdio.h>

int main() {
    FILE *arquivo;
    int numero, procurado;
    int encontrou = 0;

    printf("Digite o numero procurado: ");
    scanf("%d", &procurado);

    arquivo = fopen("numeros.txt", "r");

    if (arquivo == NULL) {
        printf("Erro ao abrir o arquivo.\n");
        return 1;
    }

    while (fscanf(arquivo, "%d", &numero) == 1) {
        if (numero == procurado) {
            encontrou = 1;
            break;
        }
    }

    if (encontrou)
        printf("Numero encontrado.\n");
    else
        printf("Numero nao encontrado.\n");

    fclose(arquivo);
    return 0;
}
```

Observe que a pesquisa não exige carregar todo o arquivo em um vetor. O programa lê um valor, compara, lê o próximo e continua até encontrar o valor ou terminar o arquivo.

4.2 Procurando uma palavra ou trecho em cada linha

Quando queremos localizar um texto dentro de uma linha, podemos combinar fgets() com strstr(). A função strstr() está declarada em string.h.

```c
#include <stdio.h>
#include <string.h>

int main() {
    FILE *arquivo;
    char linha[200];
    char termo[50];

    printf("Digite o termo procurado: ");
    scanf("%49s", termo);

    arquivo = fopen("texto.txt", "r");

    if (arquivo == NULL) {
        printf("Erro ao abrir o arquivo.\n");
        return 1;
    }
```

```
    while (fgets(linha, 200, arquivo) != NULL) {
        if (strstr(linha, termo) != NULL) {
            printf("Encontrado em: %s", linha);
        }
    }

    fclose(arquivo);
    return 0;
}
```

strstr(linha, termo) procura a sequência de caracteres de termo dentro de linha. Se encontrar, retorna um ponteiro para a ocorrência; se não encontrar, retorna NULL.

5. Escrevendo em um arquivo

Para gravar informações em um arquivo, podemos utilizar funções como fprintf(), fputs() e fputc(). Antes disso, o arquivo precisa ser aberto em um modo que permita escrita.

5.1 Criando ou sobrescrevendo com "w"

```
#include <stdio.h>

int main() {
    FILE *arquivo;

    arquivo = fopen("saida.txt", "w");

    if (arquivo == NULL) {
        printf("Erro ao criar o arquivo.\n");
        return 1;
    }

    fprintf(arquivo, "Algoritmos II\n");
    fprintf(arquivo, "Arquivos texto em C\n");

    fclose(arquivo);
    return 0;
}
```

Depois da execução, saida.txt terá:

```
Algoritmos II
Arquivos texto em C
```

Cuidado: se saida.txt já existir, o modo "w" descarta o conteúdo anterior antes da nova escrita.

5.2 Gravando valores com fprintf()

fprintf() é semelhante a printf(), mas o primeiro argumento indica o arquivo de destino.

```
int idade = 20;
float media = 8.75;

fprintf(arquivo, "Idade: %d\n", idade);
fprintf(arquivo, "Media: %.2f\n", media);
```

5.3 Gravando texto com fputs()

```
fputs("Primeira linha\n", arquivo);
fputs("Segunda linha\n", arquivo);
```

fputs() é apropriada quando queremos escrever uma sequência de caracteres sem formatação adicional.

5.4 Gravando um caractere com fputc()

```
fputc('A', arquivo);
fputc('\n', arquivo);
```

6. Acrescentando dados sem apagar o conteúdo

Quando o objetivo é inserir novas informações no final de um arquivo já existente, utilizamos o modo "a" (append). Esse modo preserva os dados anteriores.

Suponha que alunos.txt já contenha:

```
Ana 8.5
Bruno 7.0
```

O programa:

```
#include <stdio.h>

int main() {
    FILE *arquivo;
    char nome[50];
    float nota;

    printf("Nome: ");
    scanf("%49s", nome);

    printf("Nota: ");
    scanf("%f", &nota);

    arquivo = fopen("alunos.txt", "a");

    if (arquivo == NULL) {
        printf("Erro ao abrir o arquivo.\n");
        return 1;
    }

    fprintf(arquivo, "%s %.1f\n", nome, nota);

    fclose(arquivo);
    return 0;
}
```

Se forem informados Carlos e 9.0, o arquivo passará a conter:

```
Ana 8.5
Bruno 7.0
Carlos 9.0
```

Diferença essencial: "w" é indicado quando queremos produzir um novo conteúdo; "a" é indicado quando queremos acrescentar dados ao final sem apagar o que já estava armazenado.

7. Exemplo completo: cadastro simples em arquivo

O programa a seguir apresenta um pequeno menu. O usuário pode adicionar um registro ao arquivo ou listar todos os registros existentes. Esse exemplo reúne abertura, escrita, leitura e fechamento.

```
#include <stdio.h>

int main() {
    FILE *arquivo;
    int opcao;
```

```
    char nome[50];
    float nota;

    printf("1 - Adicionar aluno\n");
    printf("2 - Listar alunos\n");
    printf("Opcao: ");
    scanf("%d", &opcao);

    if (opcao == 1) {
        arquivo = fopen("alunos.txt", "a");

        if (arquivo == NULL) {
            printf("Erro ao abrir o arquivo.\n");
            return 1;
        }

        printf("Nome: ");
        scanf("%49s", nome);

        printf("Nota: ");
        scanf("%f", &nota);

        fprintf(arquivo, "%s %.1f\n", nome, nota);

        fclose(arquivo);
        printf("Registro salvo.\n");
    }
    else if (opcao == 2) {
        arquivo = fopen("alunos.txt", "r");

        if (arquivo == NULL) {
            printf("Arquivo ainda nao existe.\n");
            return 1;
        }

        while (fscanf(arquivo, "%49s %f", nome, &nota) == 2) {
            printf("%s - %.1f\n", nome, nota);
        }

        fclose(arquivo);
    }

    return 0;
}
```

8. Exemplo completo: pesquisar um registro

Agora vamos acrescentar uma pesquisa. O arquivo possui pares formados por nome e nota. O programa recebe um nome e percorre o arquivo até encontrar o registro.

```
#include <stdio.h>
#include <string.h>

int main() {
    FILE *arquivo;
    char nome[50];
    char procurado[50];
    float nota;
    int encontrou = 0;

    printf("Nome procurado: ");
    scanf("%49s", procurado);

    arquivo = fopen("alunos.txt", "r");

    if (arquivo == NULL) {
        printf("Erro ao abrir o arquivo.\n");
        return 1;
    }

    while (fscanf(arquivo, "%49s %f", nome, &nota) == 2) {
        if (strcmp(nome, procurado) == 0) {
            printf("Aluno encontrado. Nota: %.1f\n", nota);
            encontrou = 1;
            break;
        }
    }

    if (!encontrou) {
        printf("Aluno nao encontrado.\n");
    }

    fclose(arquivo);
    return 0;
}
```

strcmp() compara duas strings. Quando retorna 0, os textos são iguais. Como estamos comparando nomes armazenados em vetores de char, não devemos usar == para comparar o conteúdo textual.

8.1 Pesquisa que mostra todas as ocorrências

Se o arquivo puder conter valores repetidos e quisermos mostrar todas as ocorrências, basta não interromper o laço com break. O programa continua lendo até o fim do arquivo.

9. Trabalhando com caminhos de arquivos

Quando informamos apenas o nome, como "dados.txt", o arquivo é procurado ou criado no diretório de trabalho do programa. Também é possível informar um caminho.

```
arquivo = fopen("dados.txt", "r");
```

Em Windows, uma barra invertida dentro de uma string precisa ser representada por duas barras:

```
arquivo = fopen("C:\\dados\\alunos.txt", "r");
```

Também é comum utilizar caminhos relativos:

```
arquivo = fopen("arquivos/alunos.txt", "r");
```

Dica: quando o programa informa que não encontrou um arquivo que aparentemente existe, verifique qual é o diretório de trabalho configurado na IDE.

10. Erros comuns

10.1 Não verificar NULL

```
arquivo = fopen("dados.txt", "r");
fscanf(arquivo, "%d", &x);   /* perigoso se fopen falhar */
```

Sempre verifique se o arquivo foi aberto antes de utilizá-lo.

10.2 Usar while (!feof(arquivo)) para controlar a leitura

Evite:

```
while (!feof(arquivo)) {
    fscanf(arquivo, "%d", &x);
    printf("%d\n", x);
}
```

feof() só indica o fim do arquivo depois que uma tentativa de leitura já encontrou o final. Isso pode fazer o programa processar novamente um valor antigo ou trabalhar com uma leitura que falhou.

Prefira testar diretamente o resultado da função de leitura:

```
while (fscanf(arquivo, "%d", &x) == 1) {
    printf("%d\n", x);
}
```

10.3 Esquecer fclose()

Todo arquivo aberto e que não será mais utilizado deve ser fechado:

```
fclose(arquivo);
```

10.4 Confundir "w" com "a"

- "w": cria/sobrescreve.
- "a": acrescenta no final.

Essa diferença é fundamental quando já existem dados que precisam ser preservados.

11. Funções principais - resumo

Função	Uso	Exemplo
fopen()	Abrir/criar arquivo	fopen("dados.txt", "r")
fclose()	Fechar arquivo	fclose(arquivo)
fgetc()	Ler um caractere	fgetc(arquivo)
fgets()	Ler uma linha	fgets(linha, 200, arquivo)
fscanf()	Ler dados formatados	fscanf(arquivo, "%d", &x)
fputc()	Escrever um caractere	fputc('A', arquivo)
fputs()	Escrever texto	fputs("Texto\n", arquivo)
fprintf()	Escrever dados formatados	fprintf(arquivo, "%d", x)

12. Modelo mental

```
Programa em C
     |
     | fopen()
     v
 FILE *arquivo
     |
     +---- leitura:  fgetc / fgets / fscanf
     |
     +---- escrita:  fputc / fputs / fprintf
     |
     | fclose()
     v
Arquivo .txt
```

O ponteiro FILE *arquivo não contém todo o arquivo. Ele representa o fluxo utilizado pelo programa para acessar o arquivo aberto.

13. Exercícios de fixação

1. Leitura de números

Crie um arquivo numeros.txt contendo números inteiros. Faça um programa que leia todos os valores do arquivo e mostre a soma e a média dos números encontrados.

2. Contagem de linhas

Faça um programa que abra um arquivo texto e informe quantas linhas existem no arquivo. Utilize leitura linha por linha.

3. Pesquisa de número

Um arquivo contém uma sequência de números inteiros. Leia um valor X e informe se X aparece no arquivo e quantas vezes ele ocorre.

4. Cadastro de notas

Faça um programa que leia o nome e a nota de um aluno e acrescente essas informações ao final do arquivo notas.txt, preservando todos os registros anteriores.

5. Consulta de aluno

Considerando um arquivo em que cada linha possui o nome de um aluno e sua nota, leia um nome e procure o aluno no arquivo. Caso encontre, mostre sua nota; caso contrário, informe que o aluno não foi encontrado.

6. Filtro de valores

Um arquivo numeros.txt contém números inteiros. Crie outro arquivo chamado pares.txt contendo apenas os números pares encontrados no arquivo original.

7. Relatório

Um arquivo vendas.txt contém vários valores reais, um por linha. Leia o arquivo e crie relatorio.txt contendo o total das vendas, a média e o maior valor encontrado.

14. Checklist para resolver problemas com arquivos

- Qual arquivo será utilizado?
- O programa precisa ler, sobrescrever ou acrescentar dados?
- Qual modo de fopen() é adequado?
- O retorno de fopen() foi verificado?
- Qual função de leitura ou escrita combina melhor com o formato dos dados?
- O laço testa corretamente o sucesso da leitura?
- O arquivo foi fechado ao final?

Ideia central: arquivos permitem que os dados sobrevivam ao término do programa. Em C, o padrão é abrir o fluxo, verificar a abertura, processar os dados e fechar o arquivo.